

Machine Learning appliqué

Arbres de décision, Random Forest, XGBoost et Séries Temporelles

De l'introduction à la mise en pratique sur des données e-commerce

Support de cours – SAKPANI Paguiyedou

September 7, 2026

Contents

1	Introduction et rappels	2
1.1	Où en êtes-vous, où allons-nous	2
1.2	Apprentissage supervisé : classification vs régression	2
1.3	Vocabulaire indispensable	2
2	Préparation des données et méthodologie d'analyse	3
2.1	Le pipeline général d'un projet ML	3
2.2	Explorer les données (EDA)	3
2.3	Nettoyage et encodage	4
2.4	Séparer train/test	4
3	Les arbres de décision (Decision Trees)	6
3.1	Principe	6
3.2	Code	6
4	Random Forest	8
4.1	Principe	8
4.2	Code	8
5	XGBoost (Gradient Boosting)	10
5.1	Principe	10
5.2	Installation et code	10
6	LightGBM	12
6.1	Principe	12
6.2	Code	12
7	CatBoost	15
7.1	Principe	15
7.2	Code	15
8	Séries temporelles	17
8.1	Principe	17
8.2	Règle d'or : ne jamais mélanger le futur et le passé	17
8.3	Deux approches	17
9	Évaluer et choisir le meilleur modèle	20
9.1	Métriques de classification	20
9.2	Métriques de régression	20
9.3	Comparer plusieurs modèles proprement	20
9.4	Arbre unique vs Random Forest vs les 3 boostings : que choisir ?	21

10	Projet final – Consignes	22
10.1	Objectif	22
10.2	Données fournies	22
10.3	Travail demandé	22
10.4	Livrables attendus	22

1. Introduction et rappels

1.1 Où en êtes-vous, où allons-nous

Vous maîtrisez déjà Python de base, numpy, pandas, matplotlib/seaborn et les statistiques descriptives. Vous avez commencé le machine learning avec des modèles simples (k-NN sur iris). Ce cours vous emmène plus loin, vers les modèles *les plus utilisés en entreprise* :

- les **arbres de décision** (Decision Trees) ;
- les **forêts aléatoires** (Random Forest) ;
- le **gradient boosting** avec **XGBoost**, **LightGBM** et **CatBoost** ;
- les **séries temporelles** (prévision de ventes, de trafic, etc.).

Chaque chapitre suit le même plan : **théorie courte** → **code commenté** → **paramètres à régler pour améliorer le modèle** → **mini-projet** sur le jeu de données `dataset_exercices.csv`. Le **projet final** du chapitre 7 réunit tous ces modèles sur `dataset_projet_final.csv`.

1.2 Apprentissage supervisé : classification vs régression

- **Classification** : prédire une catégorie (le client va-t-il partir : oui/non ?).
- **Régression** : prédire une valeur numérique continue (quel sera le revenu de demain ?).

Tous les modèles de ce cours (arbre, Random Forest, XGBoost) existent en deux versions : `...Classifier` et `...Regressor`. La logique interne est la même, seule la façon de mesurer l'erreur change.

1.3 Vocabulaire indispensable

- **Feature** (variable explicative) : une colonne utilisée pour prédire.
- **Target** (cible, y) : la colonne que l'on veut prédire.
- **Overfitting** (surapprentissage) : le modèle apprend le bruit des données d'entraînement, très bon en train, mauvais en test.
- **Underfitting** (sous-apprentissage) : le modèle est trop simple, mauvais partout.
- **Hyperparamètre** : un réglage du modèle fixé *avant* l'entraînement (ex. profondeur max d'un arbre), différent d'un *paramètre appris* (ex. les seuils de coupure d'un arbre, appris automatiquement).
- **Validation croisée** (cross-validation) : découper les données en k blocs pour évaluer un modèle plusieurs fois et obtenir un score plus fiable qu'un seul split.

2. Préparation des données et méthodologie d'analyse

2.1 Le pipeline général d'un projet ML

1. **Comprendre le problème métier** : que veut-on prédire, et pourquoi ?
2. **Charger et explorer les données** (EDA – Exploratory Data Analysis).
3. **Nettoyer** : valeurs manquantes, doublons, valeurs aberrantes (outliers).
4. **Transformer** : encodage des variables catégorielles, mise à l'échelle si besoin.
5. **Séparer** train/test (et éventuellement validation).
6. **Entraîner** un modèle simple comme référence (baseline).
7. **Améliorer** : essayer d'autres modèles, régler les hyperparamètres.
8. **Évaluer et comparer** avec les bonnes métriques.
9. **Conclure** : choisir le meilleur modèle et l'interpréter.

2.2 Explorer les données (EDA)

```
1 import pandas as pd
2
3 df = pd.read_csv("dataset_exercices.csv")
4 print(df.shape)
5 print(df.info())
6 print(df.describe())           # statistiques numériques
7 print(df.isna().sum())         # valeurs manquantes par colonne
8 print(df["churn"].value_counts(normalize=True)) # équilibre des classes
```

Quel graphique pour quelle question ?

C'est souvent le point bloquant : voici une règle simple.

- Une variable numérique, distribution → histogramme (`sns.histplot`) ou boxplot (`sns.boxplot`, utile pour repérer les outliers).
- Une variable catégorielle, comptage → diagramme en barres (`sns.countplot`).
- Deux variables numériques, relation → nuage de points (`sns.scatterplot`).
- Une numérique vs une catégorielle → boxplot groupé (`sns.boxplot(x=cat, y=num)`) pour comparer les distributions.

- **Beaucoup de variables numériques entre elles** → matrice de corrélation (`sns.heatmap(df.corr(), annot=True)`).
- **Évolution dans le temps** → courbe (`plt.plot` ou `sns.lineplot(x="date", y=valeur)`).

```

1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 fig, axes = plt.subplots(1, 3, figsize=(15,4))
5 sns.histplot(df["depense_mensuelle"], kde=True, ax=axes[0])
6 sns.boxplot(x="churn", y="score_satisfaction", data=df, ax=axes[1])
7 sns.countplot(x="type_abonnement", hue="churn", data=df, ax=axes[2])
8 plt.tight_layout()
9 plt.savefig("eda_overview.png", dpi=150)
10 plt.show()

```

Astuce

Interpréter un graphique = répondre à 3 questions : (1) qu'est-ce que je regarde ? (2) est-ce que je vois une différence / une tendance / une valeur bizarre ? (3) qu'est-ce que ça implique pour le modèle (ex : classes déséquilibrées ⇒ attention à l'accuracy, il faudra regarder precision/recall) ?

2.3 Nettoyage et encodage

```

1 # Valeurs manquantes
2 df = df.dropna() # solution simple
3 # ou : df["age"] = df["age"].fillna(df["age"].median())
4
5 # Encodage des variables catégorielles (les arbres/RF/XGBoost gèrent
6 # très bien le One-Hot Encoding)
7 df_encoded = pd.get_dummies(df, columns=["genre", "region", "
8     type_abonnement"],
9                               drop_first=True)
10
11 # Séparer features (X) et cible (y)
12 X = df_encoded.drop(columns=["churn", "date_inscription"])
13 y = df_encoded["churn"]

```

Astuce contrairement à la régression linéaire ou k-NN

Les arbres de décision, Random Forest et XGBoost **n'ont pas besoin** que les variables numériques soient mises à l'échelle (StandardScaler/MinMaxScaler) : ils comparent des seuils (ex. `age <= 35`), pas des distances. Vous pouvez donc sauter cette étape pour ces 3 modèles (mais elle reste utile pour k-NN, régression logistique, SVM, réseaux de neurones).

2.4 Séparer train/test

```

1 from sklearn.model_selection import train_test_split
2
3 X_train, X_test, y_train, y_test = train_test_split(
4     X, y, test_size=0.2, random_state=42, stratify=y # stratify =
5     garder les

```

5

)

classes

proportions de

3. Les arbres de décision (Decision Trees)

3.1 Principe

Un arbre de décision pose une suite de questions du type `score_satisfaction <= 5.2 ?` pour séparer les données en groupes de plus en plus purs (qui contiennent majoritairement une seule classe). À chaque nœud, l'algorithme choisit la question qui réduit le plus **l'impureté** (mesurée par le *Gini* ou *l'entropie* en classification, par la *variance* en régression).

$$\text{Gini}(t) = 1 - \sum_{i=1}^K p_i^2 \quad \text{Entropie}(t) = - \sum_{i=1}^K p_i \log_2(p_i)$$

où p_i est la proportion de la classe i dans le nœud t . Plus le Gini/l'entropie est proche de 0, plus le nœud est pur.

3.2 Code

```
1 from sklearn.tree import DecisionTreeClassifier, plot_tree
2 from sklearn.metrics import accuracy_score, classification_report,
  confusion_matrix
3 import matplotlib.pyplot as plt
4
5 clf = DecisionTreeClassifier(max_depth=4, random_state=42)
6 clf.fit(X_train, y_train)
7
8 y_pred = clf.predict(X_test)
9 print("Accuracy :", accuracy_score(y_test, y_pred))
10 print(classification_report(y_test, y_pred))
11
12 # Tracer l'arbre lui-même (très utile pour comprendre/expliquer les dé
  cisions)
13 plt.figure(figsize=(20,10))
14 plot_tree(clf, feature_names=X.columns, class_names=["reste", "churn"],
15           filled=True, rounded=True, fontsize=8)
16 plt.savefig("arbre_decision.png", dpi=150)
17 plt.show()
18
19 # Matrice de confusion
20 from sklearn.metrics import ConfusionMatrixDisplay
21 ConfusionMatrixDisplay.from_estimator(clf, X_test, y_test)
22 plt.savefig("matrice_confusion_arbre.png", dpi=150)
23 plt.show()
```

Paramètres importants – Decision Tree

- **max_depth** : profondeur maximale. Trop grand $\Rightarrow$ overfitting ; trop petit $\Rightarrow$ underfitting. À tester avec une boucle ou **GridSearchCV**.
- **min_samples_split** : nombre minimum d'échantillons pour couper un nœud (augmenter $\Rightarrow$ arbre plus simple, moins d'overfitting).
- **min_samples_leaf** : nombre minimum d'échantillons dans une feuille.
- **criterion** : "gini" (rapide) ou "entropy" (parfois plus précis, un peu plus lent).
- **ccp_alpha** : paramètre d'élagage (pruning) qui simplifie l'arbre après entraînement pour réduire l'overfitting.

```
1 # Chercher automatiquement les meilleurs hyperparametres
2 from sklearn.model_selection import GridSearchCV
3
4 param_grid = {"max_depth": [2, 3, 4, 5, 6, 8, None],
5               "min_samples_split": [2, 5, 10, 20],
6               "criterion": ["gini", "entropy"]}
7 grid = GridSearchCV(DecisionTreeClassifier(random_state=42), param_grid,
8                   cv=5, scoring="f1")
9 grid.fit(X_train, y_train)
10 print(grid.best_params_, grid.best_score_)
```

Mini-projet – Arbre de décision

Sur `dataset_exercices.csv` :

1. Prédire **churn** avec un **DecisionTreeClassifier**. Que vaut l'accuracy avec **max_depth=2** puis **max_depth=None** ? Que se passe-t-il sur le score en train vs en test dans chaque cas ? Expliquez le lien avec l'overfitting.
2. Affichez l'arbre avec **plot_tree**. Quelle est la première question posée par l'arbre ? Est-ce cohérent avec ce que vous observez en EDA (boxplot **score_satisfaction** vs **churn**) ?
3. Utilisez **GridSearchCV** pour trouver le meilleur **max_depth**. Tracez la courbe accuracy (ou F1) en fonction de **max_depth** (une valeur par point).
4. Remplacez la cible par **depense_mensuelle** (régression) avec un **DecisionTreeRegressor**. Évaluez avec le RMSE. Que remarquez-vous par rapport à la classification ?

4. Random Forest

4.1 Principe

Un seul arbre est instable : un léger changement des données peut donner un arbre très différent (forte variance). **Random Forest** entraîne un grand nombre d'arbres différents puis **moyenne** leurs prédictions (vote majoritaire en classification, moyenne en régression). La diversité entre arbres vient de deux sources :

- **Bagging** (Bootstrap Aggregating) : chaque arbre est entraîné sur un tirage aléatoire avec remise des lignes du dataset.
- **Sélection aléatoire de features** : à chaque coupure, seul un sous-ensemble aléatoire de colonnes est considéré (`max_features`).

Résultat : un modèle beaucoup plus stable et généralement plus performant qu'un arbre seul.

4.2 Code

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 rf = RandomForestClassifier(n_estimators=300, max_depth=6,
4                             random_state=42, n_jobs=-1)
5 rf.fit(X_train, y_train)
6 y_pred = rf.predict(X_test)
7 print("Accuracy :", accuracy_score(y_test, y_pred))
8
9 # Importance des variables : quelles colonnes comptent le plus ?
10 import pandas as pd
11 importances = pd.Series(rf.feature_importances_, index=X.columns)
12 importances.sort_values().plot(kind="barh", figsize=(6,5))
13 plt.title("Importance des variables - Random Forest")
14 plt.tight_layout()
15 plt.savefig("feature_importance_rf.png", dpi=150)
16 plt.show()
```

Paramètres importants – Random Forest

- `n_estimators` : nombre d'arbres (100 à 500 en général). Plus il y en a, plus le modèle est stable, mais plus l'entraînement est lent (pas de risque d'overfitting supplémentaire en augmentant seulement ce paramètre).
- `max_depth`, `min_samples_split`, `min_samples_leaf` : mêmes rôles que pour un arbre seul, contrôlent la complexité de chaque arbre.
- `max_features` : nombre de colonnes considérées à chaque coupure ("`sqrt`" est un bon défaut en classification).

- `n_jobs=-1` : utilise tous les cœurs du processeur pour paralléliser.
- `class_weight="balanced"` : très utile quand les classes sont déséquilibrées (ex. peu de churn) pour ne pas ignorer la classe minoritaire.

Mini-projet – Random Forest

Sur `dataset_exercices.csv` :

1. Comparez un `DecisionTreeClassifier` et un `RandomForestClassifier` (mêmes `max_depth`) sur le même train/test. Lequel généralise le mieux ?
2. Tracez la courbe de l'accuracy en test en fonction de `n_estimators` (10, 50, 100, 200, 400). À partir de quand le gain devient négligeable ?
3. Affichez l'importance des variables. Le résultat confirme-t-il vos observations d'EDA sur les causes du churn ?
4. Ajoutez `class_weight="balanced"`. Le F1-score de la classe `churn=1` s'améliore-t-il ? Pourquoi est-ce important ici (regardez le déséquilibre des classes) ?

5. XGBoost (Gradient Boosting)

5.1 Principe

Random Forest construit des arbres **en parallèle** et indépendants. XGBoost (eXtreme Gradient Boosting) construit les arbres **en série** : chaque nouvel arbre essaie de corriger les erreurs (les résidus) laissées par les arbres précédents. C'est le principe du **boosting**. XGBoost ajoute en plus une régularisation (pénalité sur la complexité des arbres) qui le rend souvent très performant et robuste à l'overfitting.

$$\hat{y}_i^{(m)} = \hat{y}_i^{(m-1)} + \eta \cdot f_m(x_i)$$

où f_m est le nouvel arbre entraîné sur les erreurs, et η (`learning_rate`) contrôle la taille du pas de correction.

5.2 Installation et code

```
1 # pip install xgboost
2 from xgboost import XGBClassifier
3
4 xgb = XGBClassifier(
5     n_estimators=300, max_depth=4, learning_rate=0.05,
6     subsample=0.8, colsample_bytree=0.8,
7     eval_metric="logloss", random_state=42
8 )
9 xgb.fit(X_train, y_train,
10        eval_set=[(X_train, y_train), (X_test, y_test)],
11        verbose=False)
12
13 y_pred = xgb.predict(X_test)
14 print("Accuracy :", accuracy_score(y_test, y_pred))
15
16 # Courbe d'apprentissage (train vs test au fil des arbres ajoutés)
17 results = xgb.evals_result()
18 plt.plot(results["validation_0"]["logloss"], label="train")
19 plt.plot(results["validation_1"]["logloss"], label="test")
20 plt.xlabel("Nombre d'arbres"); plt.ylabel("Log Loss")
21 plt.legend(); plt.title("Courbe d'apprentissage XGBoost")
22 plt.savefig("courbe_apprentissage_xgb.png", dpi=150)
23 plt.show()
```

Paramètres importants – XGBoost

- `n_estimators` : nombre d'arbres ajoutés successivement.
- `learning_rate` (ou `eta`) : taille du pas (0.01 à 0.3). Plus il est petit, plus il faut

d'arbres, mais le modèle généralise souvent mieux.

- **max_depth** : profondeur de chaque arbre (souvent plus petite qu'en RF, 3 à 6, car le boosting compense avec le nombre d'arbres).
- **subsample** : fraction des lignes utilisées par arbre (évite l'overfitting).
- **colsample_bytree** : fraction des colonnes utilisées par arbre.
- **reg_alpha** / **reg_lambda** : régularisation L1 / L2 sur les poids des feuilles.
- **early_stopping_rounds** : arrête l'entraînement si le score de validation ne s'améliore plus pendant n arbres, pour éviter l'overfitting et gagner du temps.

```
1 # Early stopping : arreter des que le modele arrete de s'ameliorer sur le
  test
2 xgb = XGBClassifier(n_estimators=1000, max_depth=4, learning_rate=0.05,
3                     early_stopping_rounds=20, eval_metric="logloss",
4                     random_state=42)
5 xgb.fit(X_train, y_train, eval_set=[(X_test, y_test)], verbose=False)
6 print("Nombre d'arbres reellement utilises :", xgb.best_iteration)
```

Mini-projet – XGBoost

Sur `dataset_exercices.csv` :

1. Entraînez un `XGBClassifier` par défaut, puis comparez son F1-score à celui du Random Forest du chapitre précédent.
2. Tracez la courbe d'apprentissage (`logloss` train vs test). Le modèle overfit-il si vous mettez `n_estimators=1000` sans `early_stopping` ?
3. Utilisez `early_stopping_rounds` pour trouver automatiquement le bon nombre d'arbres. Combien d'arbres sont réellement utilisés (`best_iteration`) ?
4. Faites une petite recherche d'hyperparamètres (`learning_rate` $\in \{0.01, 0.05, 0.1, 0.3\}$) et tracez le F1-score obtenu pour chaque valeur.

6. LightGBM

6.1 Principe

LightGBM (Light Gradient Boosting Machine, développé par Microsoft) est, comme XGBoost, un algorithme de **gradient boosting** : les arbres sont construits en série, chacun corrigeant les erreurs du précédent. Sa particularité est la façon dont les arbres poussent :

- **XGBoost** construit les arbres **niveau par niveau** (*level-wise*) : tous les nœuds d'un même niveau sont coupés avant de passer au niveau suivant.
- **LightGBM** construit les arbres **feuille par feuille** (*leaf-wise*) : à chaque étape, il coupe la feuille qui réduit le plus l'erreur, même si cela rend l'arbre asymétrique. Résultat : convergence plus rapide et souvent meilleure précision, mais risque d'overfitting plus élevé sur de petits jeux de données si on ne limite pas la profondeur.

LightGBM est aussi optimisé pour être **très rapide** et **économe en mémoire** sur de gros volumes de données (histogrammes de valeurs plutôt que tri exact des points), et gère nativement les variables catégorielles sans passer par le One-Hot Encoding.

6.2 Code

```
1  # pip install lightgbm
2  from lightgbm import LGBMClassifier
3
4  lgbm = LGBMClassifier(
5      n_estimators=300, num_leaves=31, learning_rate=0.05,
6      subsample=0.8, colsample_bytree=0.8, random_state=42
7  )
8  lgbm.fit(X_train, y_train,
9          eval_set=[(X_test, y_test)])
10
11 y_pred = lgbm.predict(X_test)
12 print("Accuracy :", accuracy_score(y_test, y_pred))
13
14 # Importance des variables
15 import pandas as pd
16 importances = pd.Series(lgbm.feature_importances_, index=X.columns)
17 importances.sort_values().plot(kind="barh", figsize=(6,5))
18 plt.title("Importance des variables - LightGBM")
19 plt.tight_layout()
20 plt.savefig("feature_importance_lgbm.png", dpi=150)
21 plt.show()
```

Gérer les variables catégorielles nativement (sans get_dummies)

```
1 # LightGBM peut prendre directement des colonnes de type "category"
2 df_cat = df.copy()
3 for col in ["genre", "region", "type_abonnement"]:
4     df_cat[col] = df_cat[col].astype("category")
5
6 X_cat = df_cat.drop(columns=["churn", "date_inscription"])
7 y_cat = df_cat["churn"]
8
9 lgbm2 = LGBMClassifier(n_estimators=300, random_state=42)
10 lgbm2.fit(X_cat, y_cat, categorical_feature=["genre", "region", "
    type_abonnement"])
```

Paramètres importants – LightGBM

- **num_leaves** : nombre maximum de feuilles par arbre (équivalent, en plus direct, de **max_depth**). C'est le paramètre le plus important à régler – une valeur trop grande par rapport à la taille du dataset $\Rightarrow$ overfitting quasi garanti (règle empirique : $\text{num_leaves} < 2^{\text{max_depth}}$).
- **max_depth** : limite la profondeur en plus de **num_leaves**, utile pour contrôler l'overfitting sur peu de données (-1 = pas de limite).
- **learning_rate** : comme pour XGBoost, un pas plus petit + plus d'arbres généralise souvent mieux.
- **n_estimators** : nombre d'arbres (arrêtez-le avec de l'early stopping).
- **min_child_samples** : nombre minimum d'échantillons par feuille (monter cette valeur limite l'overfitting, très utile sur petits datasets).
- **subsample** / **colsample_bytree** : mêmes rôles que pour XGBoost.
- **categorical_feature** : liste des colonnes à traiter nativement comme catégorielles (pas besoin de **get_dummies**).

Astuce

LightGBM est en général le **plus rapide** des trois boostings de ce cours sur de gros volumes (dizaines/centaines de milliers de lignes et plus). Sur un petit dataset comme **dataset_exercices.csv** (2000 lignes), la différence de vitesse est peu visible : c'est surtout la gestion *leaf-wise* et des catégories qui change.

Mini-projet – LightGBM

Sur **dataset_exercices.csv** :

1. Entraînez un **LGBMClassifier** avec les paramètres par défaut. Comparez son F1-score à celui de XGBoost et de Random Forest (mêmes train/test).
2. Testez **num_leaves** $\in \{7, 15, 31, 63, 127\}$ et tracez le F1-score en test en fonction de **num_leaves**. À partir de quelle valeur voyez-vous l'overfitting apparaître (score train qui continue de monter, score test qui baisse) ?
3. Refaites l'entraînement en passant les colonnes catégorielles directement en type

"category" (sans `get_dummies`) via `categorical_feature`. Le résultat change-t-il ? Le temps d'entraînement change-t-il ?

4. Chronométrez (`%%time` ou `time.time()`) l'entraînement de LightGBM vs XGBoost vs Random Forest sur ce dataset. Que remarquez-vous ?

7. CatBoost

7.1 Principe

CatBoost (*Categorical Boosting*, développé par Yandex) est un troisième algorithme de gradient boosting, pensé dès le départ pour bien gérer les **variables catégorielles** (d'où son nom) sans encodage manuel, et pour réduire un biais subtil appelé *target leakage* grâce à une technique d'encodage appelée **ordered target encoding** (chaque ligne est encodée en n'utilisant que les lignes "vues avant elle" dans un ordre aléatoire, ce qui évite de tricher avec l'information de la cible). Il utilise aussi des **arbres symétriques** (oblivious trees) : à un niveau donné, la même question est posée partout dans l'arbre, ce qui le rend plus robuste au bruit et rapide à l'inférence.

7.2 Code

```
1  # pip install catboost
2  from catboost import CatBoostClassifier
3
4  # Pas besoin de get_dummies : on indique juste quelles colonnes sont
   catégorielles
5  cat_features = ["genre", "region", "type_abonnement"]
6
7  X_raw = df.drop(columns=["churn", "date_inscription"])
8  y_raw = df["churn"]
9
10 X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(
11     X_raw, y_raw, test_size=0.2, random_state=42, stratify=y_raw)
12
13 cat = CatBoostClassifier(
14     iterations=300, depth=6, learning_rate=0.05,
15     cat_features=cat_features, verbose=False, random_state=42
16 )
17 cat.fit(X_train_r, y_train_r, eval_set=(X_test_r, y_test_r))
18
19 y_pred = cat.predict(X_test_r)
20 print("Accuracy :", accuracy_score(y_test_r, y_pred))
21
22 # Courbe d'apprentissage
23 evals = cat.get_evals_result()
24 plt.plot(evals["learn"]["Logloss"], label="train")
25 plt.plot(evals["validation"]["Logloss"], label="test")
26 plt.legend(); plt.title("Courbe d'apprentissage CatBoost")
27 plt.savefig("courbe_apprentissage_catboost.png", dpi=150)
28 plt.show()
```

Paramètres importants – CatBoost

- `iterations` : équivalent de `n_estimators` (nombre d'arbres).
- `depth` : profondeur des arbres (souvent 4 à 10, les arbres étant symétriques la complexité augmente vite avec la profondeur).
- `learning_rate` : même rôle que pour XGBoost/LightGBM.
- `cat_features` : liste des colonnes catégorielles à traiter nativement – c'est le paramètre le plus important de CatBoost, à ne pas oublier.
- `l2_leaf_reg` : régularisation L2 sur les feuilles (augmenter pour réduire l'overfitting).
- `early_stopping_rounds` : comme pour XGBoost, arrête l'entraînement quand le score de validation stagne.
- `verbose` : mettre `False` ou un entier (ex. 50) pour espacer les affichages pendant l'entraînement.

Astuce

CatBoost est souvent le choix le plus simple à mettre en place quand un dataset contient **beaucoup de colonnes catégorielles** (ex. ville, catégorie produit, canal marketing) : il évite le travail d'encodage et gère bien les catégories à forte cardinalité (beaucoup de valeurs possibles), là où le One-Hot Encoding exploserait le nombre de colonnes.

Mini-projet – CatBoost

Sur `dataset_exercices.csv` :

1. Entraînez un `CatBoostClassifier` en donnant directement les colonnes catégorielles brutes (sans `get_dummies`) via `cat_features`. Comparez son F1-score à celui de LightGBM et XGBoost.
2. Testez `depth` $\in \{2, 4, 6, 8, 10\}$ et tracez le F1-score en test en fonction de la profondeur.
3. Retirez la liste `cat_features` (laissez CatBoost traiter `genre`, `region`, `type_abonnement` comme du texte libre par erreur, ou encodez-les vous-même avec `get_dummies` avant de les passer). Le score change-t-il ? Qu'est-ce que cela vous apprend sur l'intérêt de `cat_features` ?
4. **Synthèse des 3 boostings** : construisez un tableau (accuracy, F1, temps d'entraînement) comparant XGBoost, LightGBM et CatBoost sur ce dataset, et concluez lequel vous semble le plus adapté ici et pourquoi.

8. Séries temporelles

8.1 Principe

Une série temporelle est une suite de valeurs ordonnées dans le temps (ventes journalières, trafic web, etc.). Elle se décompose classiquement en :

- **Tendance** (trend) : évolution de fond à long terme.
- **Saisonnalité** (seasonality) : motif qui se répète (jour de la semaine, mois de l'année).
- **Résidu** (bruit) : ce qui reste, non expliqué.

```
1 import pandas as pd
2 from statsmodels.tsa.seasonal import seasonal_decompose
3
4 daily = df2.groupby("date")["revenu_transaction"].sum().reset_index()
5 daily = daily.set_index("date").asfreq("D").fillna(method="ffill")
6
7 decomposition = seasonal_decompose(daily["revenu_transaction"], model="
    additive",
8                                     period=7) # periode=7 pour une
                                                saisonnalite hebdo
9
10 decomposition.plot()
11 plt.savefig("decomposition_serie.png", dpi=150)
plt.show()
```

8.2 Règle d'or : ne jamais mélanger le futur et le passé

Contrairement au `train_test_split` classique (aléatoire), une série temporelle se découpe **chronologiquement** : le test doit toujours être *après* le train, jamais un mélange aléatoire (sinon le modèle voit le futur).

```
1 split_date = daily.index.max() - pd.Timedelta(days=90)
2 train = daily[daily.index <= split_date]
3 test = daily[daily.index > split_date]
```

8.3 Deux approches

1. Modèle statistique classique : ARIMA/SARIMA

```
1 from statsmodels.tsa.statespace.sarimax import SARIMAX
2
3 model = SARIMAX(train["revenu_transaction"], order=(1,1,1),
4                 seasonal_order=(1,1,1,7))
```

```

5 fit = model.fit(dispatch=False)
6 forecast = fit.forecast(steps=len(test))
7
8 plt.plot(train.index, train["revenu_transaction"], label="train")
9 plt.plot(test.index, test["revenu_transaction"], label="reel")
10 plt.plot(test.index, forecast, label="prevision SARIMA")
11 plt.legend(); plt.title("Prevision SARIMA")
12 plt.savefig("prevision_sarima.png", dpi=150)
13 plt.show()

```

Paramètres importants – ARIMA/SARIMA

- **order=(p,d,q)** : p = nombre de retards autorégressifs, d = nombre de différenciations pour rendre la série stationnaire, q = nombre de retards de la moyenne mobile.
- **seasonal_order=(P,D,Q,s)** : mêmes idées mais pour la partie saisonnière, s = longueur de la saison (7 pour hebdomadaire, 12 pour mensuel).
- On choisit souvent d et D en regardant si la série est stationnaire (test ADF) ; p et q via les graphiques ACF/PACF (plot_acf, plot_pacf).

2. Transformer le problème en ML classique (feature engineering temporel)

C'est l'approche la plus utilisée en pratique avec Random Forest / XGBoost : on crée des variables à partir de la date et des valeurs passées (lags), puis on entraîne un modèle de régression classique.

```

1 def creer_features_temporelles(data):
2     d = data.copy()
3     d["jour_semaine"] = d.index.dayofweek
4     d["mois"] = d.index.month
5     d["jour_mois"] = d.index.day
6     d["lag_1"] = d["revenu_transaction"].shift(1)      # valeur d'hier
7     d["lag_7"] = d["revenu_transaction"].shift(7)      # valeur d'il y a
8     d["moyenne_mobile_7"] = d["revenu_transaction"].shift(1).rolling(7).
9     mean()
10    return d.dropna()
11
12 data_feat = creer_features_temporelles(daily)
13 X_ts = data_feat.drop(columns=["revenu_transaction"])
14 y_ts = data_feat["revenu_transaction"]
15
16 split = int(len(X_ts)*0.85)
17 X_train_ts, X_test_ts = X_ts.iloc[:split], X_ts.iloc[split:]
18 y_train_ts, y_test_ts = y_ts.iloc[:split], y_ts.iloc[split:]
19
20 from xgboost import XGBRegressor
21 from sklearn.metrics import mean_absolute_error, mean_squared_error
22
23 reg = XGBRegressor(n_estimators=300, max_depth=4, learning_rate=0.05)
24 reg.fit(X_train_ts, y_train_ts)
25 pred = reg.predict(X_test_ts)
26 print("MAE :", mean_absolute_error(y_test_ts, pred))
27 print("RMSE :", mean_squared_error(y_test_ts, pred, squared=False))
28
29 plt.plot(y_test_ts.index, y_test_ts.values, label="reel")
30 plt.plot(y_test_ts.index, pred, label="prevision XGBoost")

```

```
30 plt.legend(); plt.title("Prevision des ventes - XGBoost")
31 plt.savefig("prevision_xgboost_ts.png", dpi=150)
32 plt.show()
```

Astuce

Les **lags** (valeurs passées) et les **moyennes mobiles** sont les features les plus puissantes en série temporelle transformée en ML. Attention : ne créez ces variables qu'à partir du **passé** (**shift(1)** minimum), jamais de la valeur du jour même, sinon vous trichez .

Mini-projet – Séries temporelles

En utilisant `dataset_exercices.csv` (colonne `date_inscription`) :

1. Agrégez le nombre d'inscriptions par jour (`groupby("date_inscription").size()`) et tracez la courbe.
2. Décomposez cette série (tendance/saisonnalité/résidu) avec `seasonal_decompose`.
3. Créez des features de lag (1, 7) et une moyenne mobile, puis entraînez un `RandomForestRegressor` pour prédire le nombre d'inscriptions du lendemain.
4. Comparez le MAE de votre modèle à un modèle naïf qui prédit simplement demain = aujourd'hui . Votre modèle fait-il mieux qu'un modèle naïf ?

9. Évaluer et choisir le meilleur modèle

9.1 Métriques de classification

- **Accuracy** : % de bonnes prédictions. Trompeur si les classes sont déséquilibrées.
- **Précision** : parmi les prédits positifs, combien le sont vraiment ?
- **Rappel (recall)** : parmi les vrais positifs, combien sont détectés ?
- **F1-score** : moyenne harmonique précision/rappel, bon résumé global.
- **AUC-ROC** : capacité du modèle à bien classer sur tous les seuils possibles.

9.2 Métriques de régression

- **MAE** : erreur moyenne absolue, facile à interpréter (même unité que y).
- **RMSE** : pénalise davantage les grosses erreurs.
- R^2 : % de variance expliquée par le modèle (1 = parfait).

9.3 Comparer plusieurs modèles proprement

```
1 from sklearn.model_selection import cross_val_score
2 from lightgbm import LGBMClassifier
3 from catboost import CatBoostClassifier
4
5 modeles = {
6     "Arbre de decision": DecisionTreeClassifier(max_depth=5, random_state
7         =42),
8     "Random Forest": RandomForestClassifier(n_estimators=300,
9         random_state=42),
10    "XGBoost": XGBClassifier(n_estimators=300, learning_rate=0.05,
11        eval_metric="logloss", random_state=42),
12    "LightGBM": LGBMClassifier(n_estimators=300, learning_rate=0.05,
13        verbose=-1, random_state=42),
14    "CatBoost": CatBoostClassifier(iterations=300, learning_rate=0.05,
15        verbose=False, random_state=42),
16 }
17
18 for nom, modele in modeles.items():
19     scores = cross_val_score(modele, X, y, cv=5, scoring="f1")
20     print(f"{nom:20s} F1 moyen = {scores.mean():.3f} (+/- {scores.std():.3f})")
```

9.4 Arbre unique vs Random Forest vs les 3 boostings : que choisir ?

- **Arbre de décision seul** : à réserver aux cas où l'**interprétabilité totale** prime (expliquer une décision à un non-technicien) ou pour un premier prototype rapide.
- **Random Forest** : bon compromis par défaut, robuste, peu de réglages nécessaires, parallélisable, tolère bien le bruit.
- **XGBoost** : souvent le plus performant toutes catégories , écosystème très mature, demande un peu plus de réglage (`learning_rate`, `early stopping`).
- **LightGBM** : à privilégier sur de **gros volumes** de données (rapidité) ou quand l'entraînement doit être répété souvent (ré-entraînement fréquent, recherche d'hyperparamètres intensive).
- **CatBoost** : à privilégier quand le dataset a **beaucoup de colonnes catégorielles**, notamment à forte cardinalité, pour éviter le travail d'encodage et le risque de fuite d'information (target leakage).

En pratique, sur un problème donné, il est courant de **tester les trois boostings** (XGBoost, LightGBM, CatBoost) avec une validation croisée et de garder celui qui donne le meilleur compromis score/temps d'entraînement pour votre cas précis – il n'y a pas de gagnant universel.

Astuce

Le meilleur modèle n'est pas toujours celui qui a le score le plus élevé : pensez aussi au **temps d'entraînement**, à la **facilité d'interprétation** (un arbre seul s'explique en une image, XGBoost beaucoup moins) et au **risque d'overfitting** si vos données de test ne représentent pas bien la réalité future.

10. Projet final – Consignes

10.1 Objectif

En équipe ou seul, vous allez analyser `dataset_projet_final.csv` (transactions e-commerce sur 3 ans) et livrer une étude complète en appliquant **tous** les modèles vus dans ce cours.

10.2 Données fournies

Le fichier `dataset_projet_final.csv` contient une ligne par transaction : `date`, `categorie`, `prix_unitaire`, `quantite`, `remise`, `canal_acquisition`, `mode_paiement`, `depense_pub`, `revenu_transaction`, `produit_retourne`.

10.3 Travail demandé

1. **Exploration** : nettoyez les données, explorez les distributions, les corrélations, l'évolution des ventes dans le temps.
2. **Classification** : prédissez `produit_retourne` avec un arbre de décision, un Random Forest, XGBoost, LightGBM et CatBoost. Comparez-les avec les bonnes métriques (le déséquilibre des classes doit être géré). Profitez de CatBoost/LightGBM pour tester la gestion native des colonnes catégorielles (`categorie`, `canal_acquisition`, `mode_paiement`) sans `get_dummies`.
3. **Régression** : prédissez `revenu_transaction` à partir des autres variables (hors date) avec les 5 mêmes familles de modèles. Comparez MAE/RMSE/ R^2 .
4. **Série temporelle** : agrégez les ventes par jour et prévoyez les ventes des 30 prochains jours avec (a) SARIMA et (b) un modèle ML avec features de lag. Comparez les deux approches.
5. **Optimisation** : pour au moins un modèle, effectuez une recherche d'hyperparamètres (`GridSearchCV` ou `RandomizedSearchCV`) et montrez le gain obtenu.
6. **Conclusion** : rédigez une synthèse (1 page) qui recommande le modèle à utiliser pour chaque tâche, avec justification (score, interprétabilité, coût de calcul).

10.4 Livrables attendus

- Un notebook (ou script) par grande partie (exploration, classification, régression, série temporelle).
- Les graphiques clés sauvegardés en image.
- Un rapport de synthèse (README ou PDF) avec vos conclusions et recommandations.

Astuce

Consultez le fichier `README.md` livré avec ce cours : il détaille, question par question, comment démarrer chaque partie du projet final.